

Проектиране и интегриране на софтуерни системи

Ивайло Андреев + gemini 3 Flash, Deepseek V3

28 юни 2026

Съдържание

1	Характеристики на разпределените софтуерни системи	2
1.1	Дефиниции	2
1.2	Основни характеристики на системите	2
1.3	Видове разпределени системи	2
1.3.1	Разпределени изчислителни системи (Distributed Computing Systems)	3
1.3.2	Разпределени информационни системи (Distributed Information Systems)	3
1.3.3	Широко разпространени / Повсеместни разпределени системи (Pervasive Distributed Systems)	3
1.4	Архитектурни тенденции	3
2	Междупроцесна комуникация (Inter-Process Communication - IPC)	4
3	Разпределени обекти и компоненти	8
3.1	Разпределени обекти	8
3.2	Разпределени компоненти	8
3.2.1	CORBA (Common Object Request Broker Architecture)	8
4	Уеб услуги (Web Services)	9
4.1	Дефиниции и архитектурни цели	9
4.2	Обобщение на жизнения цикъл на Уеб услугите	9
4.3	Сравнителен анализ: RPC, RMI и Уеб услуги	10

1 Характеристики на разпределените софтуерни системи

1.1 Дефиниции

В съвременното софтуерно инженерство (по Таненбаум и стандарта IEEE) са утвърдени следните две основни дефиниции за разпределена софтуерна система:

- Група от независими компютри, която за крайния потребител изглежда и се възприема като една единствена, цялостна система.
- Система, в която хардуерни или софтуерни компоненти, намиращи се на мрежови компютри (възли), комуникират помежду си и координират своите действия изцяло и единствено чрез изпращане и обмен на съобщения.

1.2 Основни характеристики на системите

Разпределените системи се проектират с цел постигане на следните пет ключови софтуерни архитектурни качества:

1. **Свързаност и комуникация:** Възможност за безпрепятствен обмен на ресурси и данни между отдалечените възли.
2. **Хетерогенност:** Способност на системата да работи интегрирано, независимо от различията в хардуера, операционните системи, мрежовите протоколи и езиките за програмиране.
3. **Мащабируемост (Scalability):** Капацитет на системата да реагира адекватно при значително нарастване на натоварването. Реализира се по два начина: **хоризонтално мащабиране** (*horizontal scaling*) — добавяне на нови възли към системата, и **вертикално мащабиране** (*vertical scaling*) — увеличаване на ресурсите (CPU, RAM) на съществуващите възли.
4. **Отказоустойчивост при частични откази:** Способност на архитектурата да продължи стабилното си функциониране, дори когато определени възли или комуникационни канали отпадат.
5. **Сигурност и надеждност:** Защита на данните и транзакциите при пренос по публични или споделени мрежи.

1.3 Видове разпределени системи

Разпределените софтуерни системи се класифицират в три основни категории:

1.3.1 Разпределени изчислителни системи (Distributed Computing Systems)

Използват се за тежки изчислителни задачи, изискващи изключително висока производителност. Разделят се на:

- **Клъстери (Cluster Computing):** Силно свързани (*tightly coupled*) системи. Възлите са идентични като хардуер и операционна система, физически близо един до друг, свързани в една високоскоростна локална мрежа, като една програма работи паралелно върху всички тях.
- **Гридове (Grid Computing):** Слабо свързани (*loosely coupled*) хетерогенни системи, при които хардуерът, операционните системи и мрежите на отделните възли се различават драстично. Възлите често са географски и физически силно отдалечени един от друг.

1.3.2 Разпределени информационни системи (Distributed Information Systems)

Проектирани са с цел интеграция и свързване на множество разнообразни приложения, които си комуникират по мрежата, но първоначално могат да бъдат оперативно несъвместими. Интеграцията се извършва на няколко нива:

- **Клиент-Сървър (Client-Server):** На по-ниско технологично ниво клиентите групират и изпращат заявки към сървъра, които се изпълняват по "ато-марен" начин (в тяхната цялост).
- **Peer-to-Peer (P2P):** На по-високо ниво, където всяко приложение (възел) действа едновременно като клиент и като сървър, позволявайки директна комуникация помежду им без централен посредник.

1.3.3 Широко разпространени / Повсеместни разпределени системи (Pervasive Distributed Systems)

За разлика от стабилните промишлени системи, тези системи по дефиниция са изключително нестабилни. Това се налага поради масовото навлизане на мобилни, преносими и вградени (embedded) изчислителни системи.

- Възлите в тези системи са малки, мобилни, захранват се от батерии и използват безжични комуникации.
- Характеризират се с ад-хок свързване и пълна липса на нужда от непрекъснат човешки надзор или администрация. Примери за това са умните коли, авиационните системи и IoT сензорните мрежи.

1.4 Архитектурни тенденции

- Масово навлизане на повсеместната изчислителност и IoT уреди (смарт часовници, коли, сензори).

- Постоянно разширяване на Интернет като глобална разпределена мрежа и драстично нарастване на търсенето на мултимедийни и стрийминг услуги (Webcasting).
- Доминиране на разпределени системи "по заявка" (*on-demand*) и Облачни изчисления (*Cloud Computing* като AWS, Azure, Google Cloud).

2 Междупроцесна комуникация (Inter-Process Communication - IPC)

В една разпределена софтуерна система участват **възли** (физически/виртуални машини) с хетерогенен хардуер, върху които се изпълняват процеси от даден софтуерен **компонент**. Самите компоненти могат да бъдат реализирани на всевъзможни езици за програмиране (хетерогенност). Компонентите взаимодействат помежду си, като излагат процедури за отдалечено извикване от други компоненти. Затова са необходими механизми за осъществяване на прозрачна комуникация между тях.

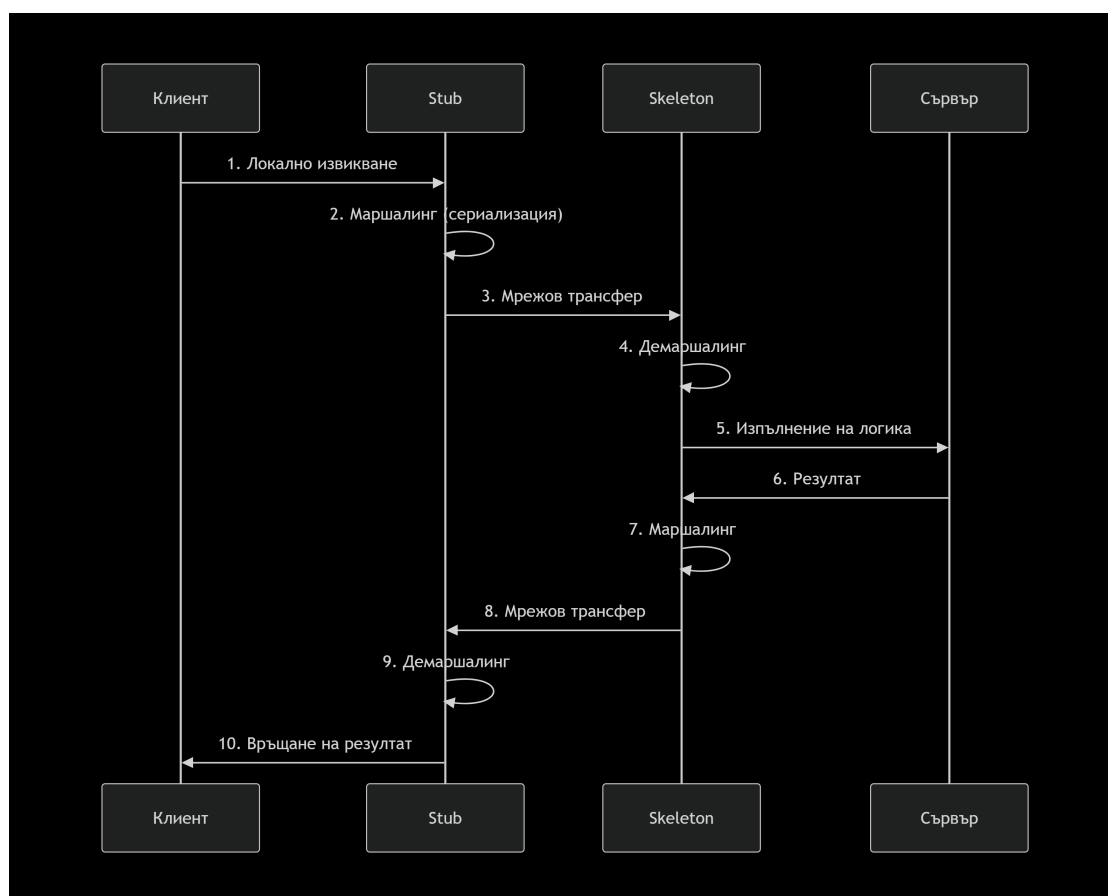
Операционните системи предоставят механизми на ниско ниво за **IPC (Inter-Process Communication)**, които включват *опашки от съобщения (message queues)*, *семафори* и *споделена памет (shared memory)*. Използването им директно обаче оставя цялата синхронизация, управление на нишки и мрежови съобщения на ниско ниво изцяло в ръцете на програмиста.

За избягване на тази сложност се използва **Middleware (междиен софтуер)**. Това е софтуерен слой, който предоставя подходящи унифицирани интерфейси, така че компонентите да си взаимодействат, скривайки детайлите на мрежата и операционната система (прозрачност на местоположението).

Чрез Middleware компонентите извикват отдалечени процедури по следния динамичен модел:

1. **Контракт и код на сървъра:** Един компонент (сървър/доставчик), който предоставя услуга, първо дефинира нейния интерфейс на **IDL (Interface Definition Language)**. След това компилаторът на Middleware превежда интерфейса от IDL към езика на сървъра (напр. C++), генерирайки име, параметри и тип на връщане на функцията. Middleware изгражда т.нар. **Skeleton (сървърен скелет)** за услугата, а програмистът попълва реалната бизнес логика на C++.
2. **Локално прокси на клиента:** Middleware предоставя интерфейса на услугата на други компоненти (клиенти/ползватели), като автоматично генерира за тях локално прокси, наречено **Stub (стъб)**, на техния език (напр. Java). Клиентският компонент вижда този Stub като локален обект/namespacespace и може да извика услугата по напълно *прозрачен* за себе си начин.

3. **Маршалинг:** При извикване на услугата от клиента, Stub-ът улавя извикването, взема подадените параметри и ги пакетира в неутрален байтов формат по мрежата (**маршалинг** / **сериализация**) към Skeleton-а на сървъра.
4. **Изпълнение и отговор:** Skeleton-ът получава пакета, разупакова го (**демаршалинг** / **десериализация**) и извиква същинската процедура на оригиналния език на сървъра (C++). След като тя приключи, Skeleton-ът взема резултата, маршализира го и го връща обратно по мрежата към Stub-а.
5. **Получаване на резултата:** Stub-ът демаршализира получения резултат и го връща на извикващия клиент като стандартна променлива на Java.



Фигура 1: Sequence diagram за процеса по отдалечено извикване

Този общ модел се реализира от процедурно-ориентираната архитектура **RPC** (**Remote Procedure Call**), както и от нейния обектно-ориентиран аналог **RMI** (**Remote Method Invocation**), специфичен за езика Java. При RMI компонентите са обекти, които запазват напълно свойствата и парадигмата на обектно-ориентираното програмиране (ООП).

RPC дефинира следните *семантики за извикване* на отдалечена процедура при възникване на мрежови грешки:

1. Maybe (Възможно е): Процедурата се извиква един път или въобще не се извиква, като извикващият не получава потвърждение за изпълнение. Това е най-ненадеждната семантика (използва се при стрийминг).
2. At-least-once (Поне веднъж): Клиентът повтаря заявката до успешно получаване на резултат. Съществува риск от дублиране и многократно изпълнение на операцията на сървъра, ако отговорите се губят в мрежата.
3. At-most-once (Най-много веднъж): Гарантира, че процедурата ще се изпълни най-много веднъж. Сървърът разпознава дублираните заявки по уникални идентификатори и не ги преизчислява, което елиминира риска от странични ефекти.
4. Exactly-once (Точно веднъж): Процедурата се изпълнява точно веднъж. Това е идеалната и най-надеждна семантика, но на практика изисква сложни транзакционни протоколи и идемпотентност.

При Java RMI се използва **RMI Registry**. То действа като централизирана директорийна услуга (телефонен указател), където доставчиците публикуват своите Stub-ове под текстови имена, а клиентите ги търсят и изтеглят локално. Това е изключително удобно, защото свалените Stub-ове се държат като native Java обекти в паметта на клиента. Също така, за разлика от RPC, RMI процедурите могат да приемат или връщат цели работещи обекти *по стойност* (чрез сериализация) или *по референция* (чрез изпращане на нов Stub).

За по-дълбоко разбиране на еволюцията на междинния софтуер (Middleware) за отдалечено извикване, в Таблица 1 са декомпозирани основните структурни и концептуални разлики между класическия RPC модел и обективно-ориентираната му имплементация в Java (RMI).

Таблица 1: Архитектурно сравнение между RPC и Java RMI

Характеристика	RPC (Remote Procedure Call)	Java RMI (Remote Method Invocation)
Архитектурен модел	Процедурно-ориентиран подход	Обектно-ориентирана парадигма (ООП)
Единица на извикване	Самостоятелни функции и процедури	Методи, капсулирани в обекти
Платформена обвързаност	Независим от езика (Language independent чрез IDL)	Тясно специализиран за Java екосистемата
Работа с параметри	Пренася само примитивни типове и структури	Поддържа цели сложни обекти <i>по стойност</i> или <i>по референция</i> (чрез нов Stub)

Мултикастът (Multicast) е метод за оптимизирано групово известяване, при който един компонент изпраща едно съобщение до множество дефинирани получатели едновременно. Логическата мрежова топология между компонентите се оформя като *дърво (Tree)* за директен достъп по най-краткия път или като *пълна мрежа (Mesh)* с алтернативни пътища за по-висока отказоустойчивост.

Рутирането в тези децентрализирани мрежи често използва **епидемични алгоритми (Gossip-Based)**. Пренасянето на съобщението се оприличава на разпространението на биологичен вирус: всеки възел, приел съобщението („заразен“), автономно пробва да го предаде на своите съседи. Информацията се разпространява експоненциално бързо. Моделът е силно устойчив на откази – ако някой възел отпадне, съобщението стига до целта по алтернативни връзки.

Основно предизвикателство при епидемичния мултикаст е изтриването на информация. Тъй като няма централен сървър, ако просто изтрием данните от един възел, неговите съседи ще му ги пратят отново при следващия си разговор. Затова се симулира „изтриване чрез модификация“ – разпространява се т.нар. **„сертификат за смърт“ (death certificate)**, който указва на всички компоненти в мрежата да игнорират и отхвърлят даденото съобщение в бъдеще.

Приложения на мултикаст:

- Разпространение на финансови данни в реално време (борсови котировки)
- Мултимедийни стрийминг услуги (IPTV, видеоконференции)
- Системи за синхронизация на клъстери
- Разпределени системи за управление на кешове

3 Разпределени обекти и компоненти

3.1 Разпределени обекти

При този дизайн архитектурните единици са разпределени обекти, които комуникират чрез RMI или механизма на разпределените събития. Те осигуряват капсулация и силна абстракция на данните.

Основни дефекти и проблеми на обектния подход:

- Интерфейсите на обектите описват само методите, но не дефинират изрично контекстните зависимости на имплементацията им.
- Програмистите са принудени ръчно да имплементират и управляват сложни системни детайли като конкурентност, нисконивова сигурност и обработка на мрежови грешки.
- Липсва автоматизирана (*out-of-the-box*) поддръжка за управление на жизнения цикъл и сложни конфигурации от обекти.

3.2 Разпределени компоненти

Проблемите на обектите налагат прехода към парадигмата на разпределените компоненти. **Софтуерният компонент** е независима градивна единица, която задължително дефинира както интерфейсите, които предоставя (*provided interfaces*), така и своите **експлицитни контекстни зависимости** (*required interfaces*). Инфраструктурата на компонентите се управлява автоматизирано от контейнери, които поемат сигурността и конкурентността. Индустриални стандарти за разпределени компоненти са **Enterprise Java Beans (EJB)** и архитектурата **CORBA**.

3.2.1 CORBA (Common Object Request Broker Architecture)

CORBA е платформено-независим стандарт за комуникация между разпределени обекти. Той решава хетерогенността, като позволява на компоненти, написани на различни езици (C++, Java, Python), да комуникират прозрачно чрез IDL-дефинирани интерфейси.

Ключови компоненти на CORBA:

- **ORB (Object Request Broker):** Посредникът, който маршрутизира заявките между обектите.
- **IDL (Interface Definition Language):** Езиково-неутрален език за описание на интерфейсите.
- **IIOP (Internet Inter-ORB Protocol):** Протоколът за комуникация между ORB-овете.
- **Naming Service:** Регистър за именуване на обекти (аналог на DNS).

4 Уеб услуги (Web Services)

4.1 Дефиниции и архитектурни цели

- **Интерфейс на уеб услуга:** Колекция от операции, изложени и достъпни по стандартизиран начин през Интернет, които се имплементират от хетерогенни ресурси (програми, бази данни).
- **Транспорт:** Уеб услугите се достъпват предимно чрез приложния протокол HTTP.
- **Цел:** Основната цел е пълно абстрахиране и изолиране на разпределеното компютърно изпълнение от специфичните детайли на имплементацията (езици за програмиране, платформи).

4.2 Обобщение на жизнения цикъл на Уеб услугите

В разпределените информационни системи **Сървърът е Процес**, който изпълнява даден софтуерен **Компонент** и притежава готови локални функции, които иска да предостави за отдалечено ползване в Интернет като хетерогенни Уеб услуги. Цялостният процес по дефиниране, откриване и изпълнение на тези услуги се затваря в следния тристъпков цикъл (*Service Triangle*), илюстриран на Фиг. 2:

1. Стъпка 1: Дефиниране и Публикуване (Publish)

За да направи функциите си достъпни за външния свят, сървърът описва техния контракт чрез **WSDL (Web Services Description Language)** – специализиран език, базиран на XML. Всеки WSDL файл се състои от две основни части (Фиг. 3):

- **Абстрактна част:** Дефинира логическия интерфейс на услугата — дефинира типовете данни (*types*), групира ги във входни и изходни съобщения (*message*) и описва самите операции и техните аргументи (*portType*).
- **Конкретна част:** Дефинира физическите детайли за достъп – мрежови адреси (*service / Endpoints*), портове и протоколи за свързване (*binding*).

След като генерира WSDL файла, сървърът го публикува в **UDDI (Universal Description, Discovery, and Integration)** – централизиран споделен регистър (хранилище), който действа като технически телефонен указател.

2. Стъпка 2: Откриване от страна на Клиента (Discover)

Клиентът (също софтуерен **процес**, намиращ се на отдалечен **възел**) се обръща към UDDI регистъра, който е разделен логически на три нива на информация (страници):

- **Бели страници (White Pages):** Дават бизнес информация **КОЙ** предоставя услугата (компанията).

- **Жълти страници (Yellow Pages):** Класифицират услугите по индустриални категории за филтриране.
- **Зелени страници (Green Pages):** Съдържат същинската техническа информация и съответния WSDL файл.

Клиентът филтрира по белите и жълтите страници, прочита зелените страници, сваля WSDL контракта и разбира конкретните адреси на сървъра. От този момент нататък UDDI регистърът повече не участва в комуникацията.

3. Стъпка 3: Директно изпълнение на услугата (Bind / Invoke)

Клиентът извиква функцията директно от сървъра в модел заявка-отговор (*Request-Response*) през HTTP протокол, управляван от специализиран **Middleware (SOAP Engine)** на двете машини (Фиг. 4):

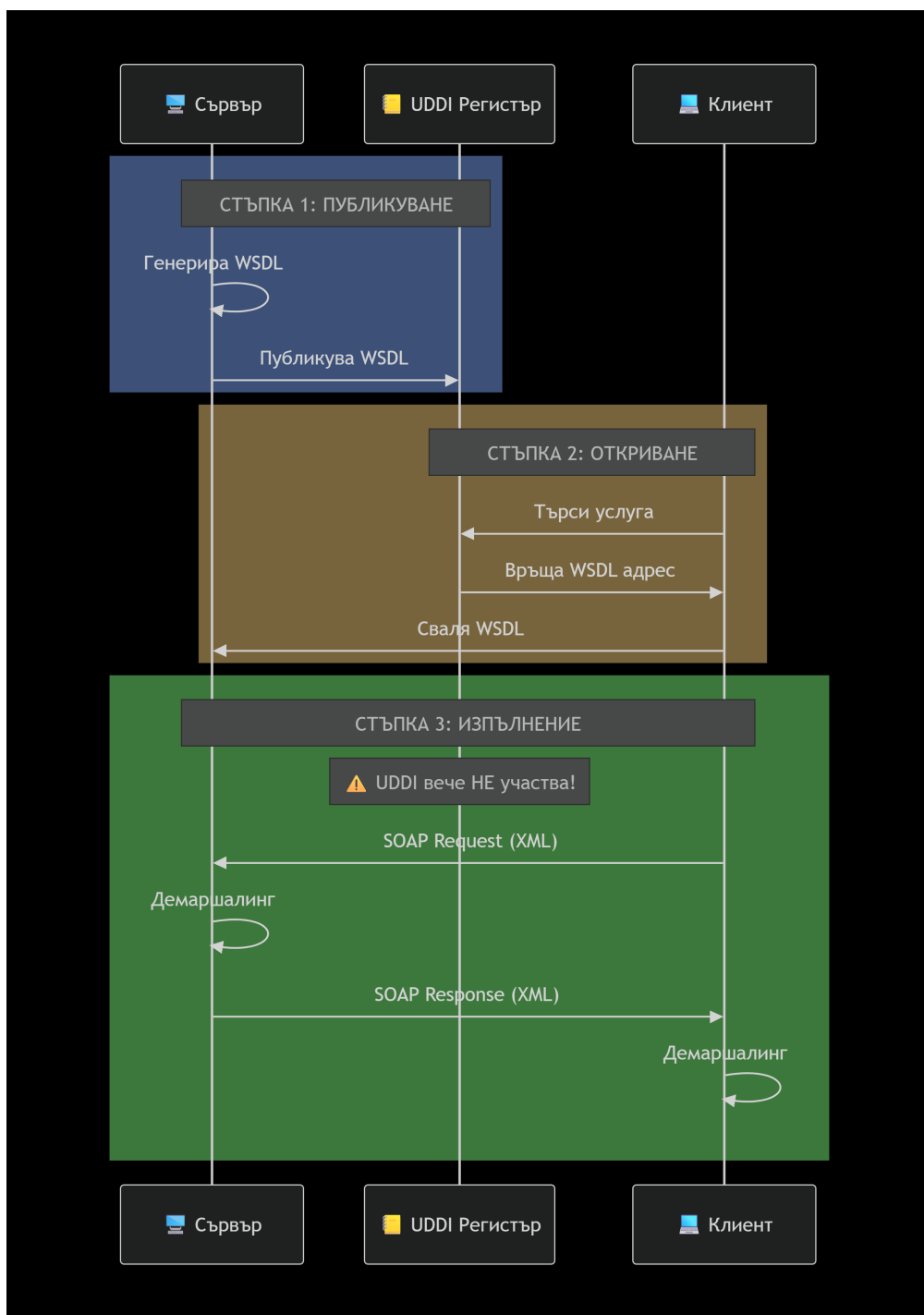
- **Заявка от клиента (Request):** Клиентският SOAP Engine взема входните параметри, капсулира ги в строго дефиниран XML обект съгласно протокола **SOAP (маршалинг)** и ги изпраща по HTTP. SOAP съобщението съдържа *Envelope* (Плик), *Header* (Хедър за метаданни) и *Body* (Тяло със същинската заявка).
- **Обработка на сървъра:** Сървърният SOAP Engine улавя XML обекта, извършва **демаршалинг** (превежда XML таговете в native променливи на своя език, напр. `int`) и извиква същинската функция на сървъра.
- **Отговор към клиента (Response):** Резултатът се маршализира обратно от сървърния енджин в нов **SOAP XML отговор** и се връща по HTTP. Клиентският енджин го демаршализира до локална променлива.

4.3 Сравнителен анализ: RPC, RMI и Уеб услуги

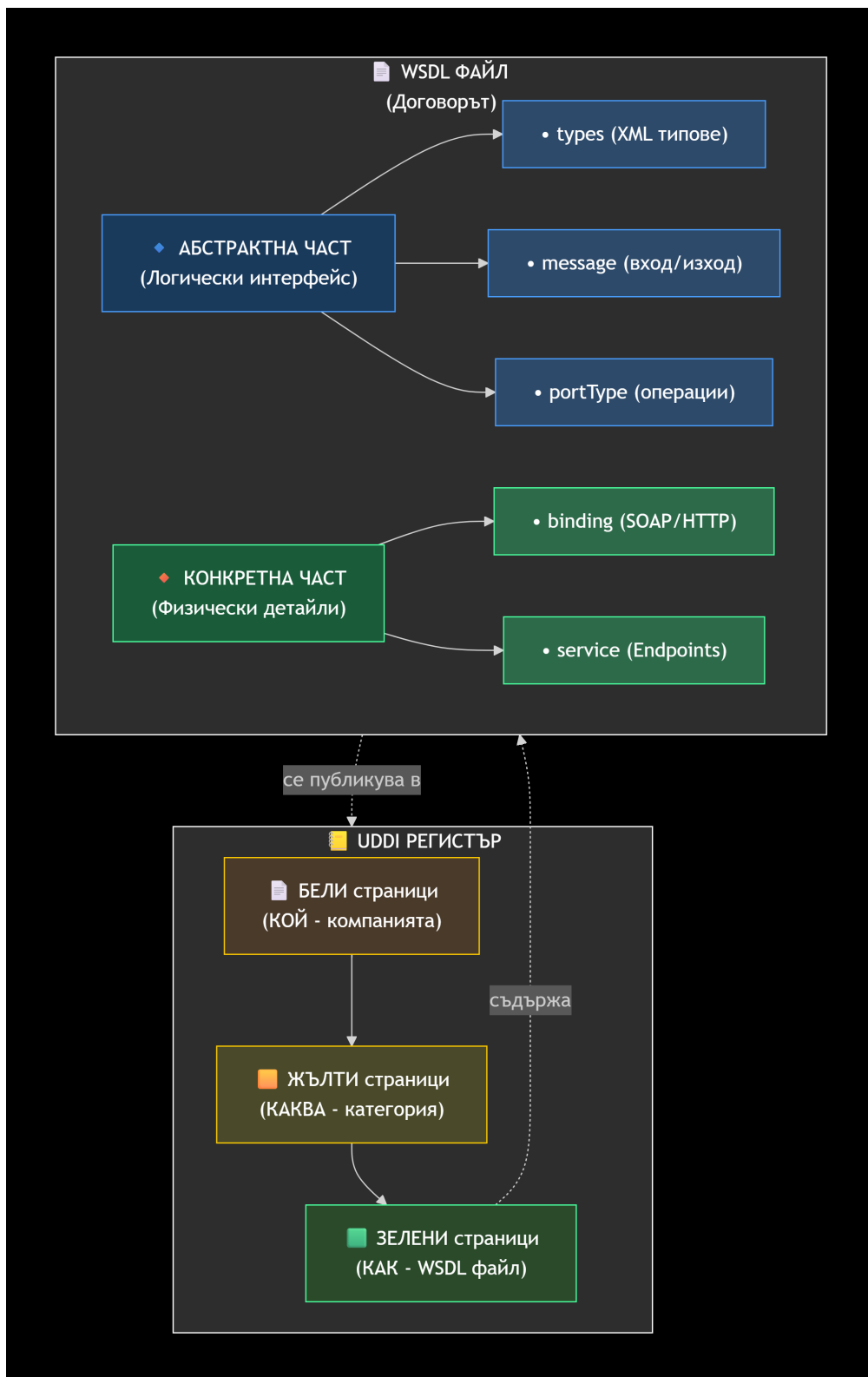
За постигане на пълна концептуална яснота, в Таблица 2 са съпоставени ключовите архитектурни характеристики на разгледаните технологии, като се използват вече дефинираните в текста компоненти и термини.

Таблица 2: Архитектурно сравнение между RPC, Java RMI и Уеб услуги

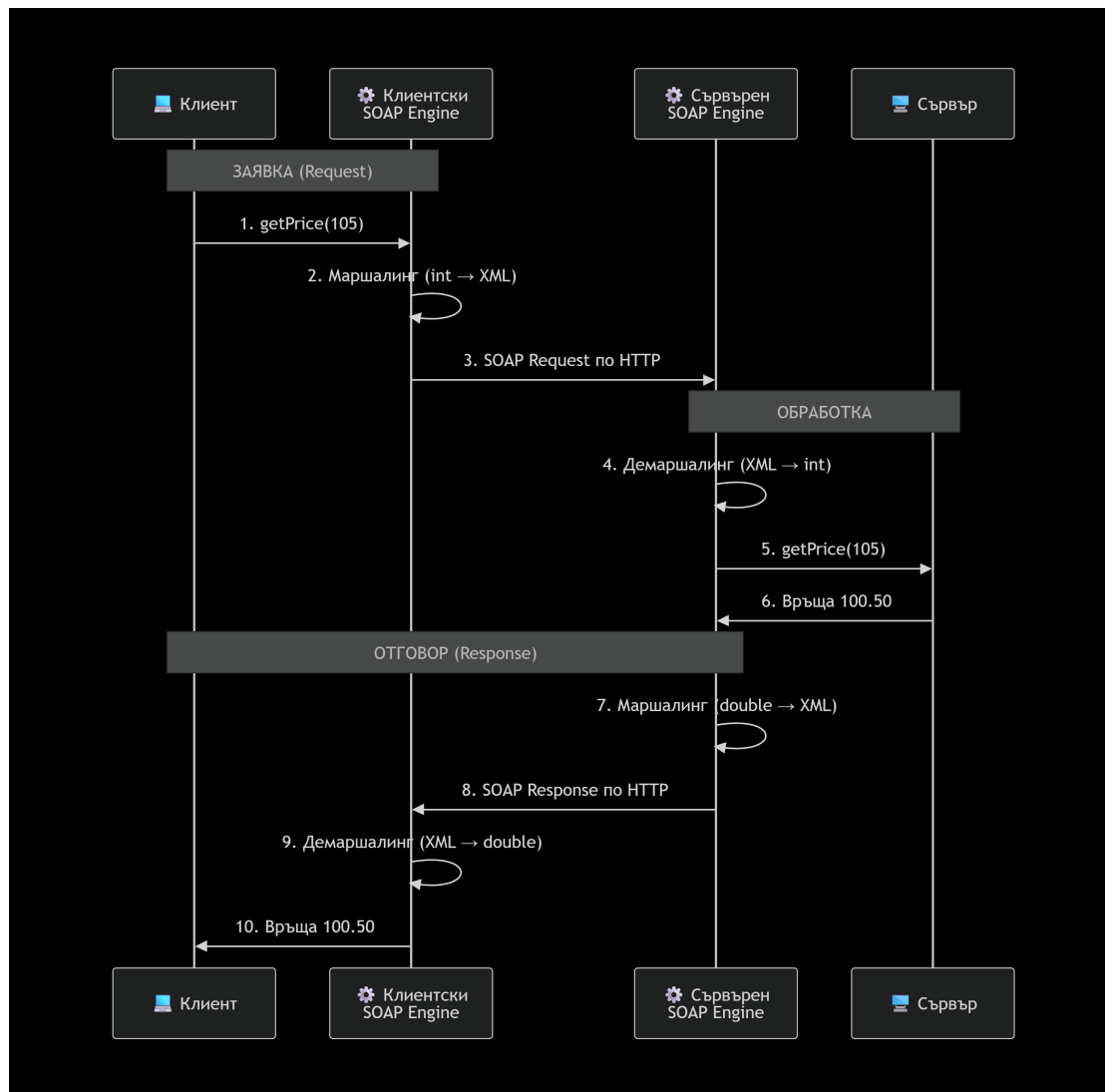
Характеристика	RPC	Java RMI	Уеб услуги
Парадигма	Процедурно-ориентиран подход	Обектно-ориентирана парадигма	Сервизно-ориентиран подход
Единица на извикване	Самостоятелни функции и процедури	Методи, капсулирани в обекти	Логически операции
Хетерогенност	Независим от езика (Language independent)	Специфичен за Java екосистемата	Хетерогенна (пълна независимост от език/ОС)
Транспорт	Обмен на съобщения по мрежата	Обмен на съобщения по мрежата	XML съобщения (SOAP) през HTTP
Работа с параметри	Примитивни типове и структури данни	Обекти по стойност или по референция (Stub)	Строго дефинирани XML типове (<i>types</i>)
Посредник	<i>Middleware</i> софтуерен слой	RMI Registry (указател за Stub-ове)	UDDI регистър + SOAP Engine
Контракт (Интерфейс)	IDL (<i>Interface Definition Language</i>)	Java интерфейс на услугата	WSDL файл (<i>Абстрактна и Конкретна част</i>)



Фигура 2: Взаимодействие в триъгълника на услугите (Публикуване, Откриване, Изпълнение)



Фигура 3: Структурна декомпозиция на WSDL файла и логическите страници в UDDI



Фигура 4: Динамика на съобщенията и маршалинг процесите през клиентския и сървърния SOAP Engine